

Reviewer Pack — Persistent Homology and Communication Complexity Lower Bound...

Entry 2e3aa0035095 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-30 07:30:02 UTC

Persistent Homology and Communication Complexity Lower Bounds

Entry ID: 2e3aa0035095 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-30 07:30:02 UTC

1. Conjecture

Field A (mathematical branch): Persistent Homology (Topological Data Analysis) **Field B** (complexity object): Communication Complexity

Statement:

The persistent homology of the communication matrix's entries (as a point cloud) has a non-trivial topological structure that correlates with the communication complexity lower bounds for the corresponding function.

Rationale (proposer's reasoning):

Topological features of the communication matrix's structure may influence the required communication, providing a new lens to analyze communication complexity via persistent homology.

Taxonomy category: LIFTING (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 4348e810489fe44b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -persistent homology communication complexity lower bounds - communication matrix point cloud topological structure - information complexity persistent homology correlation

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        n = len(A)
        for i in range(n):
            max_row = i + max(range(i, n), key=lambda j: abs(A[j][i]))
            A[i], A[max_row] = A[max_row], A[i]
            factor = 1 / A[i][i]
            A[i] = [x * factor for x in A[i]]
            for j in range(n):
                if i != j:
                    factor = A[j][i]
                    A[j] = [A[j][k] - factor * A[i][k] for k in range(n)]
        return A

    def matrix_multiplication(A, B):
        n = len(A)
        C = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def determinant(A):
        if len(A) == 1:
            return A[0][0]
        det = 0
        for c in range(len(A)):
            det += ((-1) ** c) * A[0][c] * determinant([row[:c] + row[c+1:] for row in A[1:]])
        return det
```

```

def generate_communication_matrix(n):
    return [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]

def persistent_homology(matrix):
    # Simplified version of persistent homology using determinant
    return abs(determinant(matrix))

n = random.choice([5, 10, 15, 20, 30, 40])
matrix = generate_communication_matrix(n)
homology = persistent_homology(matrix)

# Hypothetical communication complexity lower bound (simplified example)
communication_complexity = n * (n - 1) // 2

return {
    "metric_name": "Persistent Homology",
    "metric_value": homology,
    "instances_tested": 1,
    "conjecture_holds": homology > 0 and homology < communication_complexity,
    "counterexample": "" if homology > 0 and homology < communication_complexity else "mapping_undefined"
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results, preventing evaluation of support fraction or counterexamples. | next: Increase timeout duration and re-run the experiment with extended computation time

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	113607
2	propose	ollama_remote	qwen3:8b	0	0	153065
3	preregistration	ollama_remote	qwen3:8b	0	0	24293
4	novelty	ollama_remote	qwen3:8b	0	0	18195
5	novelty	ollama_remote	qwen3:8b	0	0	15196
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13613
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9148
8	judge	ollama_remote	qwen3:8b	0	0	15855

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 362972 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in research/audit/2e3aa0035095.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/2e3aa0035095.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/2e3aa0035095.tar.gz (if generated)